

Summary

C Programming: Flow Control and Selection Statements

This report summarizes the concept of flow control in C programming, which determines the order in which program statements are executed. Although C programs follow a default sequential (top-to-bottom) execution model, flow control statements allow conditional execution, repetition, and jumping between code segments.

The focus of this summary is on Selection Statements—specifically `if`, `if-else`, and `else-if` ladders—along with the definition of truth in C, scope handling using curly brackets, and practical problem-solving applications such as parity checking and finding the largest value.

1. Overview of Flow Control

In C, statements execute sequentially by default. Flow control statements alter this linear execution to meet logical and functional requirements.

Purpose of Flow Control

- **Conditional Execution**
Execute code only when specific conditions are satisfied.
- **Repetition**
Execute a statement or block multiple times.
- **Jumping**
Skip statements or move control to another part of the program.

Categories of Flow Control Statements

Category	Primary Function	Specific Statements
Selection Statements	Decision-making	<code>if</code> , <code>if-else</code> , <code>else-if</code> ladder
Iterative Statements	Repetition (loops)	<code>for</code> , <code>while</code> , <code>do-while</code>

Jump Statements

Change execution
path

`break, continue, return,`
`exit, goto`

2. Selection Statements: The `if` Statement

The `if` statement is the most basic selection mechanism in C.

Concept and Meaning

- Linguistic meaning: “If” = “**Agar**” (अगर)
- It allows execution of code **only when a condition evaluates to true**.

Syntax and Scope Rules

- **Curly Brackets { }**
 - Define the scope of the `if` block.
 - All statements inside execute if the condition is true.
- **Without Curly Brackets**
 - Only the **first statement ending with a semicolon** is controlled by the `if`.

Execution Flow

- If the condition is true → the `if` block executes.
- If the condition is false → the block is skipped and execution continues after the block.

The Concept of “Truth” in C

C evaluates conditions numerically.

- **Non-zero value → True**
 - Includes positive and negative numbers
 - Example: `-12.38` is true
- **Zero (0 or 0.0) → False**

Expressions as Conditions

Any expression that returns a value can be used as a condition:

- `i + 3`
- Function calls such as `printf()`

Example insight:

- `printf("Hello")` prints 5 characters
- Return value = 5 (non-zero)
- Therefore, the condition is **true**

3. The **if-else** and **else-if** Structures

These structures handle multiple mutually exclusive conditions.

The **if-else** Statement

The **else** block represents an “**Otherwise**” (अदरवाइज) case.

Key Characteristics

- Exactly **one block executes**
- Both blocks **cannot** execute
- One block **must** execute if **else** exists
- **else:**
 - Cannot exist independently
 - Must immediately follow the **if** block

The **else-if** Ladder

Used when more than two conditions are required.

Evaluation Rules

- Conditions are evaluated **top to bottom**
- As soon as one condition is true:
 - Its block executes
 - Remaining conditions are skipped

Final **else** Behavior

- **With final else**
 - Exactly one block executes
- **Without final else**
 - At most one block executes
 - It is possible that no block executes if all conditions are false

4. Practical Logic Applications

Parity Checking (Even or Odd)

Two methods are discussed for checking whether a number n is even.

Method 1: Modulus Operator

if ($n \% 2 == 0$)

- If remainder after division by 2 is zero $\rightarrow$ number is even

Method 2: Bitwise Logic

- Even number $\rightarrow$ Least Significant Bit (LSB) is 0
- Odd number $\rightarrow$ LSB is 1

if ($(n \& 1) == 0$)

- Bitwise AND with 1 identifies even numbers

Finding the Largest of Three Numbers

Let the numbers be A , B , and C .

Approach 1: Independent **if** Statements

- Check:
 - $A > B \ \&\& \ A > C$
 - $B > A \ \&\& \ B > C$
 - $C > A \ \&\& \ C > B$
- Inefficient because all conditions are checked even after finding the largest value

Approach 2: Nested **if-else** (Optimized)

1. Check if $A > B \ \&\& \ A > C$
 - If true $\rightarrow A$ is largest
2. Else:
 - Compare B and C
 - The larger value is selected

This approach reduces comparisons and demonstrates nesting, where **if-else** blocks are placed inside other conditional blocks.